

LAPORAN PROGRES

From Ticket to Deployment

Mapping the Software Delivery Lifecycle Across 8 Eras of AI Capability & Autonomy

1. Peran Anggota Tim

Tanggung Jawab	Alissa Dyah Ayu Permatasari 25/575323/PPA/07279	Elsa Aiziyah 25/572243/PPA/07187	Farichaturrifqi Aryanitasari 25/574224/PPA/07240	Muhammad Fikry Rizal 25/573936/PPA/07229	Muhammad Zidan Arsyad (25/572381/PPA/07192)
Perumusan problem & studi literatur	V	V	V	V	V
Ilustrasi data & perhitungan komputasi	V	V	V	V	V
Desain arsitektur sistem	V	V	V	V	V
Implementasi metode AI/ML/DL	V	V	V	V	V

2. Deskripsi Problem

Proses penanganan tiket perangkat lunak, mulai dari feedback pengguna hingga deployment perbaikan, pada banyak organisasi masih bergantung pada intervensi manual di hampir setiap tahap. Berdasarkan pipeline 9 tahap yang dipetakan pada deck sebelumnya (Feedback Capture → Feedback Loop), teridentifikasi empat masalah utama:

- Long time-to-resolution: proses triase manual (klasifikasi kategori, severity, dan routing) memakan waktu karena bergantung pada penilaian subjektif engineer dan pencarian keyword yang tidak presisi (Era 1–2).
- Duplicate ticket rework: tiket yang sebenarnya sudah pernah dilaporkan tidak terdeteksi karena pencocokan hanya berbasis kata kunci persis, sehingga tim menghabiskan effort untuk investigasi berulang.
- Human bottleneck pada setiap gate approval (start sprint, PR merge, deployment) diperlukan, tetapi jika tidak didukung ringkasan/analisis otomatis, approval menjadi lambat terutama saat volume tiket tinggi.
- Inconsistent quality control: acceptance criteria, reproduction steps, dan release notes yang ditulis manual rentan tidak lengkap atau tidak konsisten antar tiket.

Tujuan proyek ini adalah merancang sistem intelligent agent yang mengotomasi sebagian besar tahapan pipeline tersebut, triase, investigasi, perencanaan, implementasi, verifikasi, hingga deployment, sambil mempertahankan human checkpoint pada titik keputusan berisiko tinggi. Peningkatan otonomi dilakukan bertahap, mengikuti kerangka “Capability Expansion” (Uncertainty & Context, Tools & Automation, Multi-Role Collaboration, Business Ownership) yang telah dipetakan ke 8 era AI pada deck sebelumnya.

3. Ilustrasi Data & Perhitungan Komputasi

Contoh berikut mengilustrasikan dua komputasi inti yang dipakai pada tahap Smart Triage: (a) deteksi duplikasi tiket berbasis kemiripan vektor, dan (b) klasifikasi severity berbasis probabilitas.

3.1 Deteksi Tiket Duplikat (Cosine Similarity)

Misalkan tiga tiket direpresentasikan sebagai vektor fitur (hasil embedding/TF-IDF sederhana) atas 5 dimensi terkait topik {crash, login, tombol UI, lambat, timeout}:

Tiket	Deskripsi Singkat	Vektor Fitur
A	“Aplikasi crash saat login, koneksi timeout”	[0.80, 0.10, 0.00, 0.30, 0.20]
B	“Login gagal, aplikasi force-close, sering timeout”	[0.75, 0.15, 0.00, 0.35, 0.25]
C	“Tombol ‘Simpan’ di halaman profil tidak berfungsi”	[0.10, 0.00, 0.90, 0.05, 0.00]

Kemiripan Tiket A vs Tiket B:

$$\cos(A, B) = \frac{0.8 \cdot 0.75 + 0.1 \cdot 0.15 + 0 \cdot 0 + 0.3 \cdot 0.35 + 0.2 \cdot 0.25}{0.8832 \times 0.8775} = \frac{0.770}{0.7750} \approx 0.994$$

Skor 0.994 berada jauh di atas ambang batas duplikasi ($\tau = 0.85$), sehingga Tiket B ditandai sebagai kemungkinan duplikat dari Tiket A dan tidak dibuatkan tiket investigasi baru.

Kemiripan Tiket A vs Tiket C:

$$\cos(A, C) = \frac{0.8 \cdot 0.1 + 0.1 \cdot 0 + 0 \cdot 0.9 + 0.3 \cdot 0.05 + 0.2 \cdot 0}{0.8832 \times 0.9069} = \frac{0.095}{0.8010} \approx 0.119$$

Skor 0.119 jauh di bawah ambang batas, sehingga Tiket C diproses sebagai tiket baru yang berdiri sendiri.

3.2 Klasifikasi Severity (Softmax)

Model klasifikasi menghasilkan skor mentah (logit) untuk 4 kelas severity dari satu tiket masuk:

Kelas Severity	Logit (z)	e_r	Probabilitas
Low	1.2	3.320	22.9%
Medium	0.5	1.649	11.4%
High	2.1	8.166	56.4%
Critical	0.3	1.350	9.3%

Total $\Sigma e_r = 14.485$. Karena $P(\text{High}) = 56.4\%$ adalah probabilitas tertinggi, tiket diklasifikasikan sebagai High dan dirutekan ke antrian prioritas tinggi. Jika probabilitas tertinggi berada di bawah ambang keyakinan tertentu (misal $< 40\%$), sistem mengeskalasi ke human checkpoint alih-alih memutuskan sendiri, sejalan dengan gate approval pada Era 4–5.

4. Rumus dan Notasi Matematika

4.1 Cosine Similarity

$$\cos(\theta) = \frac{A \cdot B}{\|A\| \|B\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum A_i^2} \cdot \sqrt{\sum B_i^2}}$$

Diketahui :

- A, B = vektor representasi (embedding/TF-IDF) dari dua tiket yang dibandingkan.
- A_i, B_i = komponen (nilai fitur) ke- i dari masing-masing vektor; n = jumlah dimensi vektor.
- $\|A\|, \|B\|$ = norma Euclidean (panjang) vektor A dan B .
- θ = sudut antara dua vektor; $\cos(\theta)$ mendekati 1 berarti dua tiket sangat mirip (kandidat duplikat), mendekati 0 berarti tidak terkait.

4.2 Softmax (Klasifikasi Severity)

$$P(y = k|x) = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}, \text{ untuk } j = 1, 2, \dots, K$$

Diketahui:

- z_k = skor mentah (logit) yang dihasilkan model untuk kelas ke- k .
- K = jumlah total kelas (di sini 4: Low, Medium, High, Critical); penjumlahan pada penyebut berjalan dari kelas $j = 1$ sampai K .
- $P(y = k|x)$ = probabilitas tiket x tergolong kelas k ; seluruh probabilitas berjumlah 1.
- e = basis logaritma natural (≈ 2.718), digunakan agar seluruh skor bernilai positif dan proporsional.

4.3 Aturan Keputusan Otonomi (Gate Rule)

Aksi = Otonom, jika $P(y = k|x) \geq \tau$ dan $\text{risk}(k) \leq R_{\text{maks}}$; selain itu $\rightarrow$ Eskalasi ke Human

- τ (tau) = ambang batas keyakinan minimum agar agent boleh bertindak tanpa persetujuan manusia.
- $\text{risk}(k)$ = estimasi risiko/dampak dari kelas k (mis. severity Critical selalu berisiko tinggi walau confidence tinggi).
- R_{maks} = batas risiko maksimum yang masih boleh dieksekusi otomatis; nilai ini yang menentukan baris “Required Human Approval” pada tabel Autonomy Boundary & Governance.

5. Penelitian dengan Topik Serupa

Tiga penelitian berikut membahas otomasi tahap-tahap yang setara dengan Smart Triage dan Investigation pada pipeline ini, menggunakan pendekatan machine learning / deep learning:

- [1] He, J., Xu, L., Yan, M., Xia, X., & Lei, Y. (2020). Duplicate Bug Report Detection Using Dual-Channel Convolutional Neural Networks. *Proceedings of the 28th International Conference on Program Comprehension (ICPC 2020)*.

Penelitian ini merepresentasikan pasangan bug report sebagai matriks dua kanal, lalu memakai CNN untuk menangkap hubungan semantik antara keduanya dan mengklasifikasikan apakah pasangan itu duplikat. Evaluasi pada tiga proyek open source, yaitu OpenOffice, Eclipse, dan NetBeans, menghasilkan akurasi 0,9429, 0,9685, dan 0,9534, serta 0,9552 pada dataset gabungan

- [2] Isotani, H., et al. (2023). Sentence Embedding and Fine-Tuning to Automatically Identify Duplicate Bugs. *Frontiers in Computer Science*, 4:1032452.

Sistem yang mereka bangun memakai Sentence-BERT untuk mengubah tiap elemen bug report menjadi vektor, lalu menghitung kemiripan semantik antar laporan dari kemiripan vektornya. SBERT tersebut di-fine-tune dengan teks bug report agar sesuai domain. Pendekatan ini menjadi dasar teknik cosine similarity pada Bagian 3.1.

- [3] Ramay, W. Y., Umer, Q., Yin, X. C., Zhu, C., & Illahi, I. (2019). Deep Neural Network-Based Severity Prediction of Bug Reports. *IEEE Access*, 7, 46846–46857.

Penelitian ini memproses teks bug report dengan teknik NLP, menghitung skor emosi untuk tiap laporan, menyusun vektor dari laporan yang sudah diproses, lalu meneruskan vektor dan skor emosi itu ke klasifikator berbasis deep neural network untuk memprediksi severity. Pendekatan tersebut mendasari klasifikasi severity pada Bagian 3.2.

Intelligent Agent System

6. Single-Agent vs. Multi-Agent: Justifikasi Pemilihan

Pipeline “Ticket to Deployment” terdiri atas 9 tahap yang membutuhkan domain keahlian dan tools yang sangat berbeda. Triase memerlukan NLP, akses log/observability digunakan untuk investigasi, code generation untuk implementasi, testing untuk verifikasi, hingga API deployment untuk rilis. Berdasarkan karakteristik ini, kelompok memilih arsitektur multi-agent (bukan single-agent monolitik) dengan alasan:

- Spesialisasi mengurangi context-overload: setiap agent hanya perlu memuat konteks dan tools yang relevan dengan tahapnya, mengurangi risiko instruksi yang tercampur/hilang dibanding satu agent yang menangani seluruh pipeline.
- Mengurangi cascading hallucination: penelitian MetaGPT (Hong et al., 2024) menunjukkan bahwa chaining LLM secara naif menyebabkan halusinasi yang menumpuk antar-langkah; pembagian peran dengan Standard Operating Procedure (SOP) yang jelas terbukti menekan masalah ini.
- Paralelisasi & throughput: beberapa tiket dapat diproses bersamaan oleh instance agent yang berbeda (mis. beberapa Investigation Agent berjalan paralel), sesuai dengan diagram Era 7 pada deck sebelumnya.

- Governance granular: setiap agent role dapat memiliki batas otonomi berbeda (lihat tabel Autonomy Boundary & Governance), Investigation Agent bisa otonom penuh, sementara Deploy Agent wajib menunggu human sign-off untuk rilis berdampak besar.

Untuk skala yang lebih kecil atau tugas yang berdiri sendiri (mis. hanya memperbaiki satu bug dari satu issue GitHub), pendekatan single-agent seperti SWE-agent (Yang et al., 2024) tetap relevan karena overhead koordinasi antar-agent lebih rendah. Pipeline pada proyek ini memilih multi-agent karena cakupannya mencakup keseluruhan siklus (triase → deployment → monitoring), bukan satu tugas coding tunggal.

7. Penelitian yang Menggunakan Intelligent Agent

- [1] Hong, S., Zheng, X., Chen, J. P., Cheng, Y., et al. (2024). MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework. ICLR 2024.

MetaGPT menyandikan prosedur operasi standar tim perangkat lunak ke dalam urutan prompt, dengan peran seperti Product Manager, Architect, dan Engineer. Ketika dipasangkan dengan GPT-4, sistem ini mencapai Pass@1 sebesar 85,9% pada HumanEval dan 87,7% pada MBPP. Pada eksperimen mereka dengan dataset SoftwareDev, MetaGPT mencapai tingkat penyelesaian tugas 100%.

- [2] Qian, C., Liu, W., Liu, H., Chen, N., et al. (2024). ChatDev: Communicative Agents for Software Development. ACL 2024, 15174-15186.

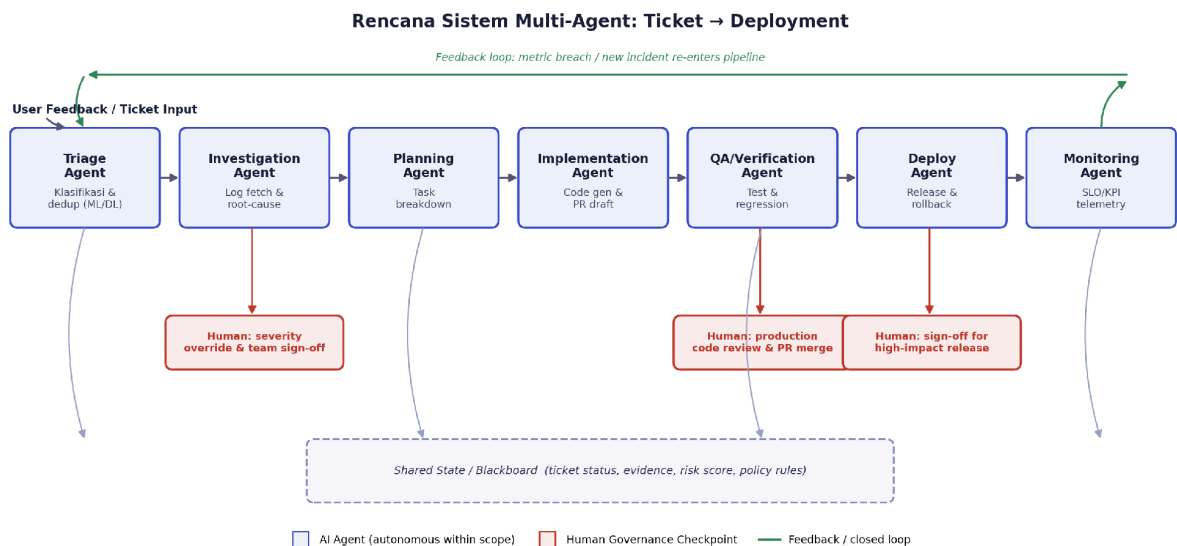
ChatDev menempatkan agent dengan peran CEO, CTO, Programmer, Reviewer, dan Tester untuk berkolaborasi lewat chat chain yang menirukan model waterfall, mencakup desain, koding, dan pengujian.

- [3] Yang, J., Jimenez, C. E., Wettig, A., Lieret, K., Yao, S., Narasimhan, K., & Press, O. (2024). SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. arXiv:2405.15793.

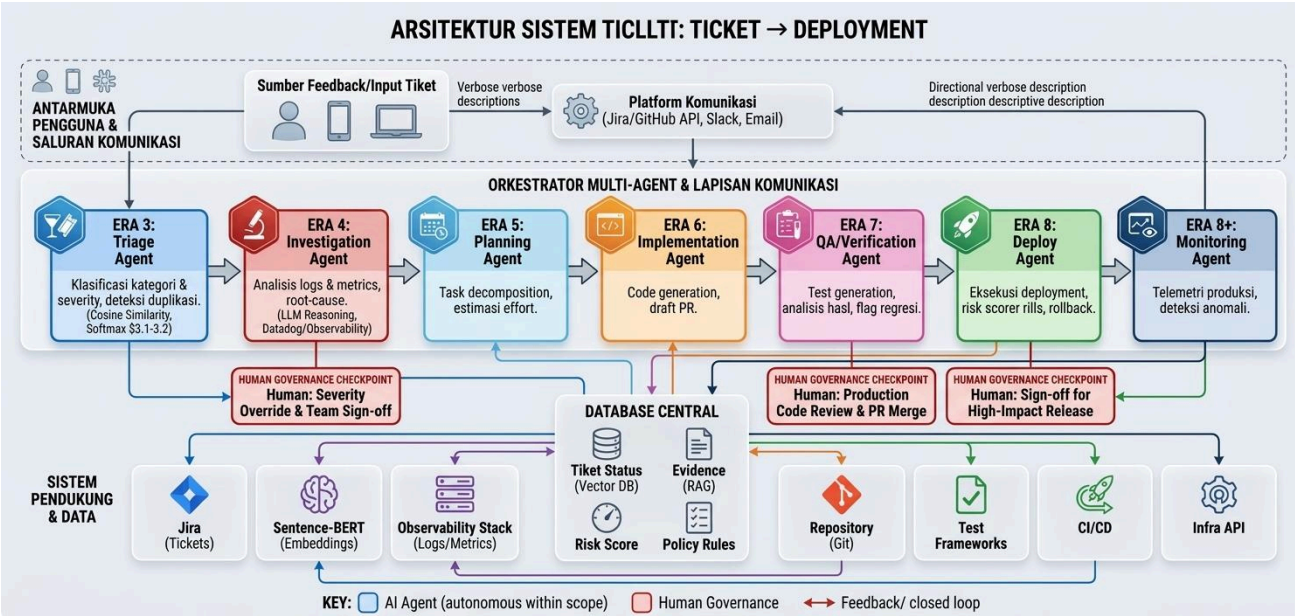
SWE-agent adalah single agent berbasis LLM yang memakai Agent-Computer Interface (ACI) untuk membaca, mengedit, dan mengeksekusi kode guna menyelesaikan issue pada repositori GitHub nyata melalui benchmark SWE-bench

8. Diagram Rencana Sistem

Diagram berikut menggambarkan rencana arsitektur multi-agent untuk pipeline Ticket → Deployment, termasuk shared state (blackboard), human governance checkpoint, dan feedback loop dari Monitoring Agent kembali ke Triage Agent.



Gambar 1. Rencana Sistem Multi-Agent: Ticket Deployment



Gambar 2. Aristektur Sistem Ticket Deployment

9. Komponen Internal Setiap Agent

Setiap agent pada diagram di atas dibangun dari lima komponen kanonik arsitektur agen cerdas: (1) Perception/Input, cara agent menerima data; (2) Memory/State, apa yang diingat agent antar langkah; (3) Reasoning/Planning, “otak” pengambil keputusan; (4) Tool-use/Action Interface, alat yang bisa dipanggil agent; (5) Output, hasil yang diteruskan ke agent berikutnya atau ke manusia. Penerapannya per agent:

Agent	Input/Perception	Memory/State	Reasoning Core	Tools	Output
Triage	Teks/gambar tiket mentah	Riwayat tiket serupa (vector DB)	Classifier ML/DL + skor kemiripan (§3.1–3.2)	Jira/GitHub API, vector search	Kategori, severity, tim tujuan

Agent	Input/Perception	Memory/State	Reasoning Core	Tools	Output
Investigation	Tiket + bukti awal	Log historis, basis pengetahuan insiden	LLM reasoning berantai (root-cause)	Log fetch API, Datadog/observability	Langkah reproduksi, hipotesis akar masalah
Planning	Laporan investigasi	Status backlog/sprint	Task decomposition planner	Jira/Project mgmt API	Subtask, estimasi effort
Implementation	Spesifikasi task	Konteks kode (RAG atas repo)	LLM code-generation	Git, file ops, test runner	Code diff / draft PR
QA/Verification	PR + spesifikasi test	Riwayat hasil test	Test generation & analisis hasil	CI/CD API, test framework	Laporan pass/fail, flag regresi
Deploy	PR yang disetujui	Riwayat deployment & rollback	Risk scorer rilis	CI/CD deploy pipeline, infra API	Eksekusi deployment, trigger rollback
Monitoring	Telemetry produksi	Baseline SLO/SLA	Deteksi anomali	Observability stack	Alert, tiket feedback baru

10. Justifikasi Metode AI / ML / DL

Pemilihan metode pada setiap tahap mengikuti kerangka “Capability Expansion” (Era 1–8) pada deck sebelumnya: metode dipilih berdasarkan trade-off akurasi, explainability, dan biaya komputasi, bukan otomatis memakai LLM/agent di semua tempat.

- Rule-based / Traditional AI (Era 1), tetap dipertahankan untuk validasi field wajib dan SLA timer, karena butuh determinism dan audit trail yang mudah dijelaskan ke auditor/compliance.
- Machine Learning klasik (Era 2, mis. Random Forest/SVM), dipakai untuk fitur tabular yang jelas seperti prediksi churn/impact likelihood, karena data terstruktur, cepat dilatih, dan mudah diinterpretasi.
- Deep Learning (Era 3, CNN/BERT/Sentence-BERT), dipilih untuk representasi teks tidak terstruktur (deteksi duplikasi, klasifikasi severity) karena mampu menangkap kemiripan semantik yang tidak tertangkap keyword matching, terbukti pada literatur Bagian 5 (akurasi 85–99%).
- Generative AI / LLM (Era 4), untuk tugas yang butuh menghasilkan bahasa alami atau kode: ringkasan tiket, draft acceptance criteria, release notes, dan code generation.
- Task-Driven & Multi-Agent System (Era 5–7), untuk orkestrasi lintas-tahap dengan tools dan state bersama; terbukti pada MetaGPT/ChatDev/SWE-agent meningkatkan task completion rate dibanding satu kali pemanggilan LLM.

Kesimpulan: pendekatan yang digunakan bersifat hybrid dan bertahap (progresif per era), bukan lompat langsung ke Era 8 (Autonomous Enterprise Operations), sejalan dengan roadmap kematangan (Maturity Rollout Roadmap) dan tabel Autonomy Boundary & Governance yang sudah disusun pada progres sebelumnya.